

TP4 - Semana 4 - Base de Datos II (Unidad 2)

Reportes analiticos asistidos por IA sobre Food Store: joins, subconsultas, agregacion y ventana.

Base de trabajo: foodstore_tp3 (copia de trabajo en PostgreSQL 17, localhost:5432), poblada masivamente al cierre de la Semana 3 (producto: 50.002 - cliente: 20.001 - pedido: 200.001 - linea_pedido: 600.002). Metricas con EXPLAIN ANALYZE (3 corridas).

Contexto y adaptacion al modelo real

La presentacion usa un esquema generico (usuario, detalle_pedido, pedido.total, eliminado). El modelo Food Store real difiere: cliente en vez de usuario; linea_pedido en vez de detalle_pedido; el total se calcula como SUM(cantidad * precio_unitario) de linea_pedido (no hay columna total en pedido); el borrado logico es activo = TRUE en cliente, producto y categoria, y pedido no tiene columna de baja. Todas las consultas se adaptaron y verificaron sobre el modelo real.

El archivo queries.sql mencionado en la consigna no se encuentra en el repositorio de trabajo. Se tomaron las dos consultas analiticas del tipo sugerido por el enunciado (historial de ventas por producto y top de pedidos recientes con agregacion), que cruzan tres tablas.

Parte 1 - Consultas analiticas lentas

QA: Ventas de un producto (historial). Cruza linea_pedido, pedido y producto (3 tablas); filtra por producto_id = 100 y devuelve las ultimas ventas.

```
SELECT p.id, p.fecha, pr.nombre AS producto,
       lp.cantidad, lp.cantidad * lp.precio_unitario AS subtotal
FROM   linea_pedido lp
JOIN   pedido p ON p.id = lp.pedido_id
JOIN   producto pr ON pr.id = lp.producto_id
WHERE  lp.producto_id = 100
ORDER  BY p.fecha DESC
LIMIT 50;
```

QB: Top 100 pedidos recientes con cliente y total. Cruza pedido, cliente y linea_pedido; agrega el total de cada pedido y ordena por fecha.

```
SELECT p.id, p.fecha, c.nombre || ' ' || c.apellido AS cliente,
       SUM(lp.cantidad * lp.precio_unitario) AS total
FROM   pedido p
JOIN   cliente c ON c.id = p.cliente_id
JOIN   linea_pedido lp ON lp.pedido_id = p.id
GROUP  BY p.id, p.fecha, c.nombre, c.apellido
ORDER  BY p.fecha DESC
LIMIT 100;
```

Planes ANTES (medicion inicial)

QA - antes (39.538 ms): el optimizador eligio un Nested Loop mantenido, pero la tabla combinada se barrio con un Parallel Seq Scan (Rows Removed by Filter: 199.997 por worker).

```
Limit (cost=8620.89..8622.06 rows=10) (actual time=39.386 rows=12)
-> Gather Merge (Workers Launched: 2)
    -> Sort (Sort Key: p.fecha DESC) (actual time=12.248 rows=4)
        -> Nested Loop (actual 12.212 ms, 4 filas/worker)
            -> [Nested Loop]
                -> Parallel Seq Scan on linea_pedido lp
                    Filter: (producto_id = 100)
                    Rows Removed by Filter: 199997
                -> Index Scan using pedido_pkey on pedido p
            -> Index Scan using producto_pkey on producto pr
Planning Time: 18.215 ms | Execution Time: 39.538 ms
```

QB - antes (687.412 ms): el plan cruza TODAS las lineas con un Merge Join por la PK y agrega 600.002 filas antes del LIMIT.

```
Limit (actual 686.808 ms rows=100)
-> Sort (top-N heapsort, 45kB) rows=200001
    -> GroupAggregate (600.002 filas de entrada, 200.001 de salida)
        -> Incremental Sort (Presorted Key: p.id)
            -> Merge Join Merge Cond: (p.id = lp.pedido_id)
                -> Nested Loop + Memoize (pedido x cliente_pkey)
                -> Index Scan linea_pedido_pkey (600.002 filas)
Planning Time: 10.242 ms | Execution Time: 687.412 ms
```

Propuesta de la IA y justificacion por nodo

QA: el cuello no es el join (Nested Loop correcto) sino el barrido completo de linea_pedido para filtrar un solo producto. La IA propuso **CREATE INDEX idx_linea_pedido_producto ON linea_pedido (producto_id)**: convierte el nodo Parallel Seq Scan en un Bitmap Index Scan de las ~12 filas del producto. No cambia el algoritmo de join (sigue Nested Loop), si el metodo de acceso de la tabla externa.

QB: el LIMIT 100 se aplica al final; el Merge Join y el GroupAggregate recorren 600.002 lineas. La IA propuso reescribir la consulta para empujar el LIMIT hacia adentro, seleccionando primero los 100 pedidos mas recientes por idx_pedido_fecha y agregando solo sus lineas (Nested Loop por linea_pedido_pkey). No se crea indice nuevo.

```
CREATE INDEX idx_linea_pedido_producto ON linea_pedido (producto_id);
```

```
-- QB reescrita (sin indice nuevo):
SELECT t.id, t.fecha, c.nombre || ' ' || c.apellido AS cliente,
       SUM(lp.cantidad * lp.precio_unitario) AS total
FROM   (SELECT id, fecha, cliente_id FROM pedido
        ORDER BY fecha DESC, id DESC LIMIT 100) t
JOIN   cliente c ON c.id = t.cliente_id
JOIN   linea_pedido lp ON lp.pedido_id = t.id
GROUP BY t.id, t.fecha, c.nombre, c.apellido
ORDER BY t.fecha DESC, t.id DESC;
```

Planes DESPUES (medicion final)

QA - despues (0.446 ms): el Parallel Seq Scan fue reemplazado por un Bitmap Index Scan sobre el nuevo indice.

```
Limit (actual 0.362 ms rows=12)
-> Sort (Sort Key: p.fecha DESC) (quicksort 25kB)
    -> Nested Loop (actual 0.327 ms, 12 filas)
        -> Index Scan using producto_pkey on producto pr
        -> Nested Loop
            -> Bitmap Heap Scan on linea_pedido lp
                Recheck Cond: (producto_id = 100)
                -> Bitmap Index Scan on idx_linea_pedido_producto
            -> Index Scan using pedido_pkey on pedido p
Planning Time: 7.005 ms | Execution Time: 0.446 ms
```

QB - despues (6.194 ms): el plan pasa de Merge Join + GroupAggregate de 600k filas a un Hash Join (cliente) + Nested Loop (lineas de los 100 pedidos) + GroupAggregate de 299 filas.

```
GroupAggregate (actual 6.111 ms rows=100)
-> Sort (299 filas, quicksort 44kB)
    -> Nested Loop (299 filas)
        -> Hash Join Hash Cond: (c.id = pedido.cliente_id)
            -> Seq Scan on cliente c
            -> Hash
                -> Limit (Index Scan Backward using idx_pedido_fecha)
        -> Index Scan using linea_pedido_pkey on linea_pedido lp
Planning Time: 1.947 ms | Execution Time: 6.194 ms
```

Tabla 1.2 - Resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (despues)	Mejora
QA - Ventas de un producto	Nested Loop (con Parallel Seq Scan de linea_pedido: elimina 199.997 filas por worker)	CREATE INDEX idx_linea_pedido_producto ON linea_pedido(producto_id)	Nested Loop (con Bitmap Index Scan del nuevo indice + pedido_pkey)	39,538 -> 0,446 ms (x88,7)
QB - Top 100 pedidos recientes + total	Merge Join pedido<->linea + Nested Loop + Memoize (cliente); GroupAggregate de 600.002 filas	Reescritura limit-first (subconsulta ORDER BY fecha DESC LIMIT 100 con idx_pedido_fecha)	Hash Join pedido<->cliente + Nested Loop por PK de las 100 lineas; GroupAggregate de 299 filas	687,412 -> 6,194 ms (x111,0)

Criterio de aceptacion (cambio de algoritmo): en QB el JOIN PRINCIPAL cambio de Merge Join a Nested Loop: sin indice que remarque el orden, el Merge tenia que barajar 600.002 lineas contra 200.001 pedidos y agruparlas antes de llegar al LIMIT; la reescritura selecciona primero los 100 pedidos y solo entonces junta sus lineas por PK, tocando 299 filas y ordenando 299 (memoria 44 kB). En QA el algoritmo de join no cambio (Nested Loop correcto para un filtro muy selectivo); el cambio fue en el metodo de acceso del nodo externo (seq scan total -> bitmap index), que es lo que eliminaba el costo. La equivalencia de resultados de la reescritura de QB se comprobo comparando los 100 ids antes y despues: 0 diferencias.

Parte 2 - Lectura critica de un plan interpretado por IA

Se tomo el plan real QB-antes de la Parte 1, que tiene dos nodos de join (Merge Join y Nested Loop + Memoize). La IA lo explico nodo por nodo y se contrasto cada afirmacion contra el plan.

Afirmaciones de la IA y verificación

Afirmacion de la IA	Correcta ?	Correccion / evidencia del plan real
En el Nested Loop de la rama izquierda, la tabla externa es pedido (200.001 filas) y la interna es cliente, resuelta por su PK en cada iteracion.	Si	El plan muestra Nested Loop rows=200001: externa = Index Scan pedido_pkey (200.001 filas), interna = Index Scan cliente_pkey con Index Cond: (id = p.cliente_id). La Memoize cachea los clientes (Cache Hits: 180000, Misses: 20001), evitando releer la PK 200.000 veces.
El join principal es un Merge Join entre pedido y linea_pedido por pedido_id, viable porque ambos lados llegan ordenados por su PK.	Si	Merge Cond: (p.id = lp.pedido_id). Entradas: Index Scan pedido_pkey e Index Scan linea_pedido_pkey (orden por PK), por eso el Merge no necesita ordenar para la clave.
Como el SELECT tiene LIMIT 100, el plan solo procesa 100 filas de las tablas grandes.	No	El Merge Join y el GroupAggregate recorren las 600.002 lineas y las 200.001 filas de pedido por completo (GroupAggregate: 600.002 entrada / 200.001 salida, actual 665,909 ms). El LIMIT actua al final (Sort top-N heapsort, 45 kB). Por eso la reescritura que empuja el LIMIT hacia adentro fue la mejora real (6,194 ms).
El costo de Sort = 122.845 es el tiempo total de la consulta en milisegundos.	No	El costo 122.845 es la unidad de costo estimada (abstracta) del planner, no milisegundos: el Execution Time real fue 687,412 ms. Además, el costo estimado de un nodo intermedio no debe confundirse con el tiempo medido de la consulta completa.

Parte 3 - Consultas resumen, rankings y subconsultas

Spec (a): ranking con funcion de ventana

Generar una consulta SQL sobre el esquema Food Store que devuelva, para cada cliente vigente (cliente.activo = TRUE) con al menos un pedido, su id, nombre, apellido, el total gastado (suma de cantidad x precio_unitario de linea_pedido) y su puesto en un ranking de mayor a menor gasto. En caso de empate, los clientes comparten el puesto y se dejan huecos (funcion RANK). Desempate final por id. Sin SELECT *. Se pide una segunda version con estructura distinta (sin la funcion RANK) que resuelva exactamente la misma pregunta.

```
-- V1 (RANK):
SELECT c.id, c.nombre, c.apellido,
       SUM(lp.cantidad * lp.precio_unitario) AS gasto,
       RANK() OVER (ORDER BY SUM(lp.cantidad * lp.precio_unitario) DESC) AS puesto
FROM   cliente c
JOIN   pedido p      ON p.cliente_id = c.id
JOIN   linea_pedido lp ON lp.pedido_id = p.id
WHERE  c.activo = TRUE
GROUP BY c.id, c.nombre, c.apellido
ORDER BY puesto, c.id;

-- V2 (ROW_NUMBER + MIN de ventana, sin RANK):
WITH gastos AS (
  SELECT c.id, c.nombre, c.apellido,
         SUM(lp.cantidad * lp.precio_unitario) AS gasto
  FROM   cliente c
  JOIN   pedido p      ON p.cliente_id = c.id
  JOIN   linea_pedido lp ON lp.pedido_id = p.id
  WHERE  c.activo = TRUE
  GROUP BY c.id, c.nombre, c.apellido),
r AS (
  SELECT g.*, ROW_NUMBER() OVER (ORDER BY g.gasto DESC, g.id) AS rn
  FROM   gastos g)
SELECT r.id, r.nombre, r.apellido, r.gasto,
       MIN(r.rn) OVER (PARTITION BY r.gasto) AS puesto
FROM   r
ORDER BY puesto, r.id;
```

La primera version que propuso la IA para V2 contaba COUNT(DISTINCT gasto) mayor o igual al propio (equivalente a DENSE, no a RANK) y era O(n²) sobre 20.001 clientes: se descarto por semantica y por rendimiento. La V2 final usa ROW_NUMBER + MIN por grupo de empate, que reproduce RANK para los empates: el puesto es la posicion de la primera fila del grupo (comparte puesto y deja huecos).

Spec (b): subconsulta correlacionada

Generar una consulta SQL que devuelva, para cada pedido del cliente id = 6 que tenga al menos una linea de pedido, su id, fecha y el total del pedido (suma de cantidad x precio_unitario). Orden por fecha ascendente y luego por id. La especificacion fija el filtro "al menos una linea" porque una subconsulta escalar correlacionada devolveria el pedido aun sin lineas (total NULL), mientras que el join + GROUP BY no lo haria: sin fijarlo, las versiones no serian equivalentes. Se

pide una segunda version con estructura distinta (join + agregacion).

```
-- V1 (subconsulta escalar correlacionada):
SELECT p.id, p.fecha,
       (SELECT SUM(lp.cantidad * lp.precio_unitario)
        FROM   linea_pedido lp WHERE lp.pedido_id = p.id) AS total
FROM   pedido p
WHERE  p.cliente_id = 6
AND    EXISTS (SELECT 1 FROM linea_pedido lp WHERE lp.pedido_id = p.id)
ORDER BY p.fecha, p.id;

-- V2 (join + agregacion):
SELECT p.id, p.fecha,
       SUM(lp.cantidad * lp.precio_unitario) AS total
FROM   pedido p
JOIN   linea_pedido lp ON lp.pedido_id = p.id
WHERE  p.cliente_id = 6
GROUP BY p.id, p.fecha
ORDER BY p.fecha, p.id;
```

Verificacion de equivalencia

```
-- (a) ranking
SELECT count(*) FROM ( (consulta_V1_a) EXCEPT (consulta_V2_a) ) d; -- 0
SELECT count(*) FROM ( (consulta_V2_a) EXCEPT (consulta_V1_a) ) d; -- 0
-- (b) total por pedido
SELECT count(*) FROM ( (consulta_V1_b) EXCEPT (consulta_V2_b) ) d; -- 0
SELECT count(*) FROM ( (consulta_V2_b) EXCEPT (consulta_V1_b) ) d; -- 0
```

Verificacion	Resultado	Conclusion
(a) V1 EXCEPT V2 / V2 EXCEPT V1	0 filas / 0 filas	Ranking formalmente equivalente (RANK vs ROW_NUMBER+MIN)
(b) V1 EXCEPT V2 / V2 EXCEPT V1	0 filas / 0 filas	Total por pedido equivalente (correlacionada vs join+GROUP BY)

Muestras de resultado (V1)

```
-- Ranking V1 (top 5 de 20.001 clientes):
id | cliente | gasto | puesto
7430 | Nombre30 Apellido30 | 1.374.029,00 | 1
7914 | Nombre14 Apellido14 | 1.353.384,00 | 2
63 | Nombre63 Apellido63 | 1.323.221,00 | 3
8043 | Nombre43 Apellido43 | 1.315.121,00 | 4
3176 | Nombre76 Apellido176 | 1.314.224,00 | 5

-- Spec (b) pedidos del cliente 6 (10 filas):
id | fecha | total
100012 | 2025-01-19 | 122.379,00
140014 | 2025-01-20 | 129.154,00
60010 | 2025-02-22 | 94.646,00
80011 | 2025-02-23 | 131.443,00
180016 | 2025-05-14 | 72.899,00
160015 | 2025-06-06 | 43.354,00
120013 | 2025-06-13 | 54.167,00
7 | 2025-06-14 | 125.800,00
20008 | 2025-08-05 | 30.343,00
40009 | 2025-08-16 | 25.569,00
```

Parte 4 - Registro de la competencia

Consulta comun entregada a todos los equipos (facturacion por categoria, con dos JOIN y una agregacion):

```
SELECT ca.nombre AS categoria,
       ROUND(SUM(lp.cantidad * lp.precio_unitario)::numeric, 2) AS facturacion
FROM   linea_pedido lp
JOIN   producto pr ON pr.id = lp.producto_id
JOIN   categoria ca ON ca.id = pr.categoria_id
WHERE  ca.activo = TRUE AND pr.activo = TRUE
GROUP BY ca.nombre
ORDER BY facturacion DESC
LIMIT 10;

-- ANTES (sin indice nuevo): Hash Join + Hash Join + HashAggregate
Limit (actual 188.866 ms) -> Finalize GroupAggregate (rows=2)
-> Hash Join (pr.categoria_id = ca.id)
-> Hash Join (lp.producto_id = pr.id)
-> Parallel Seq Scan on linea_pedido (200.001 filas/worker)
Execution Time: 189.274 ms

-- DESPUES (con idx_linea_pedido_producto): plan identico
```

Limit -> Finalize GroupAggregate -> Hash Join -> Hash Join
-> Parallel Seq Scan on linea_pedido (igual)
Execution Time: 181.186 ms

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo despues (ms)	Mejora (x)
Nuestro (Plan A)	Indexar linea_pedido(producto_id) para la facturacion por categoria	189,274	181,186	1,04 - SIN MEJORA: el agregado barre las 600.000 lineas completas; el indice no reduce el volumen. Propuesta DESCARTADA con datos.
Nuestro (Plan B, de la Parte 1)	Reescritura limit-first sobre ranking de pedidos recientes (top N)	687,412	6,194	x111 - usada y documentada en la Parte 1

Punto de la competencia (aplicar solo lo que se puede justificar en el plan real): el indice que habia servido en la Parte 1 (QA) NO mejora la facturacion por categoria porque el plan real muestra que el cuello es el barrido total de linea_pedido en el Parallel Seq Scan y el costo de los dos Hash Joins sobre 600.002 filas; para ese barrido no hay indice puntual que reduzca el volumen. La mejora real de la sesion vino de reescrituras que reducen filas antes de agregar (LIMIT hacia adentro), tal como quedo medido en la Parte 1 (x111).

DUIA - Declaracion de Uso de IA

Herramienta	Para que se uso	Prompt / spec (resumen)	Se acepto / se descarto - por que
OpenCode	Analizar los planes EXPLAIN ANALYZE de QA y QB	"Este es el plan; que nodo domina el costo y que propuesta aplicarias"	Aceptado: identifico el nodo Parallel Seq Scan de QA y el Merge Join sobre 600k filas de QB.
OpenCode	Proponer optimizacion para QA	Proponer indices para el filtro producto_id (nodo scan del Nested Loop)	Aceptado: CREATE INDEX idx_linea_pedido_producto -> x88,7 (el algoritmo de join no cambio; cambio el metodo de acceso del nodo externo).
OpenCode	Proponer optimizacion para QB	Reescribir para aprovechar ORDER BY + LIMIT con idx_pedido_fecha	Aceptado: reescritura limit-first -> x111, verificada equivalente (ids 100/100, 0 diferencias).
OpenCode	Alternativa de ranking para spec (a)	RANK con COUNT(DISTINCT gasto >= propio)	DESCARTADO: semantica DENSE con empates (no RANK) y O(n2). Se reemplazo por ROW_NUMBER + MIN(PARTITION BY gasto): RANK exacto y eficiente.
OpenCode	Competencia - indexar la consulta de facturacion	Indice sobre linea_pedido(producto_id) para facturacion por categoria	DESCARTADO: midio 189,274 -> 181,186 ms (x1,04, plan identico); un agregado de barrido total no se acelera con indice puntual.

Entregables: tablas de las Partes 1-4, SQL de carga de cambios e interna del plan, consultas Parte 3 y verificaciones en los scripts 01_explain_antes.sql, 02_cambios_propuestos.sql, 03_explain_despues.sql, 04_consulta_competencia.sql y 05_consultas_parte3.sql (carpeta UNIDAD2\TP4). Defensa oral: cualquier indice o reescritura de esta entrega se explica sobre el plan real.